

Capitolo 5b - Advanced CPU Scheduling

Status Done

Scheduling dei Thread

Nei moderni SO che supportano i thread, lo scheduling non avviene a livello di processo, ma a livello di thread del kernel.

- **Thread del kernel:** il kernel è consapevole della loro esistenza ed è responsabile della loro esecuzione sulla CPU fisica.
- **Thread Utente:** sono gestiti da una libreria (a livello utente); il kernel non è consapevole della loro esistenza. Per essere eseguiti, devono essere associati a un processo leggero (lwp), che è a sua volta associato a un thread del kernel.

Ambito della contesa

L'ambito della contesa definisce quali thread competono per l'uso della CPU e dove avviene la pianificazione.

1. Ambito della contesa ristretto al processo (PCS)

- *Dove avviene:* Lo scheduling avviene all'interno del processo.
- *Chi contende:* I thread a livello utente competono tra loro per aggiudicarsi un lwp libero.
- *Chi schedula:* la libreria dei thread pianifica i thread utente su lwp.
- *Modelli SO:* usato nei modelli da molti a uno e da molti a molti.
- *Pianificazione:* solitamente basata su **priorità** (il programmatore le stabilisce). È applicata la **prelazione** (un thread a priorità più alta interrompe l'esecuzione di uno a priorità più bassa), ma non è garantito il *time slicing* se le priorità sono uguali.

2. Ambito della contesa allargato al sistema (SCS)

- *Dove avviene:* lo scheduling avviene a livello di tutto il sistema.
- *Chi contende:* tutti i thread a livello kernel del sistema competono per l'uso delle CPU fisiche.
- *Chi schedula:* il kernel esamina tutti i thread del sistema per determinare quale eseguire.
- *Modelli SO:* usato nei modelli da uno a uno (es. windows, linux).

Scheduling Pthreads (API POSIX)

L'API Pthread consente ai programmatori di specificare l'ambito della contesa desiderato al momento della creazione del thread.

1. Valori di contesa

Pthreads utilizza due valori per specificare l'ambito:

- `pthread_scope_process` : pianifica i thread con scheduling PCS.
- `pthread_scope_system` : pianifica i thread con scheduling SCS.

2. Funzioni API

L'API Pthread offre funzioni per leggere e impostare l'ambito della contesa:

Funzione API	Scopo	Parametri Chiave
<code>pthread_attr_setscope</code>	Imposta l'ambito della contesa.	Puntatore agli attributi del thread (<code>attr</code>), Valore dell'ambito (<code>scope</code>).
<code>pthread_attr_getscope</code>	Appura (legge) l'ambito della contesa corrente.	Puntatore agli attributi del thread (<code>attr</code>), Puntatore a un intero che riceve il valore (<code>scope</code>).

Scheduling per Sistemi Multiprocessore

Lo scheduling della CPU diventa molto più complesso quando sono disponibili più unità di elaborazione (multiprocessori). Sebbene in passato il termine MP indicasse più chip fisici, oggi si applica a una varietà di architetture, tra cui **CPU multicore**, **Core multithread**, sistemi **NUMA** e sistemi **eterogenei**.

Quando i processori sono **omogenei** (identici in funzionalità), qualsiasi CPU può eseguire qualsiasi thread presente nella coda.

Esistono due strategie principali per gestire le decisioni di scheduling in un sistema multiprocessore:

1. Multielaborazione asimmetrica

- **Meccanismo:** un singolo processore (il **Master Server**) gestisce **tutte le decisioni** di scheduling, elaborazione I/O e attività di sistema. Gli altri processori eseguono solo codice utente.
- **Vantaggio:** semplifica la gestione, riducendo la necessità di **condividere dati** sulle strutture di sistema.
- **Svantaggio:** il Master Server è un potenziale collo di bottiglia.

2. Multielaborazione Simmetrica (SMP)

- **Meccanismo:** ciascun processore è **autogestito** e ha il proprio scheduler che seleziona un thread da eseguire. È l'approccio standard per i sistemi MP moderni.
- **Organizzazione delle Code (Ready Queues):**
 1. **Coda comune:** tutti i thread sono in un'unica coda condivisa. Richiede **lock** per prevenire *race condition* (due processori che scelgono lo stesso thread), ma il lock può diventare un collo di bottiglia per le prestazioni.
 2. **Code private:** ogni processore ha la sua coda privata di thread. Questo è l'approccio **più comune** sui sistemi SMP perché elimina i problemi di prestazioni della coda condivisa e favorisce l'efficienza della memoria cache.

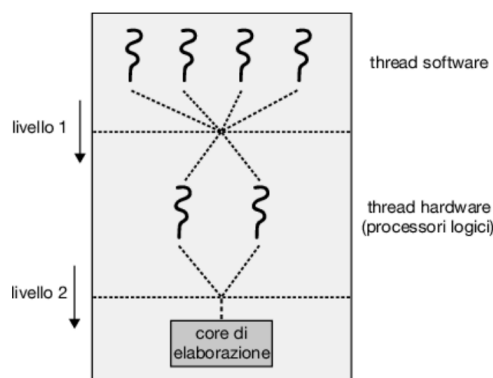
Sfide delle architetture moderne

I sistemi moderni introducono nuove complessità che vanno oltre la semplice assegnazione di un thread a una CPU.

1. Processori multicore e multithread (CMT / SMT):

L'hw è progettato per recuperare il tempo perso in attesa dei dati (stallo della memoria):

- **Problema: Memory Stall**
Si verifica quando un processore trascorre tempo significativo (fino al 50%) in attesa che i dati siano disponibili dalla memoria (spesso perché i dati richiesti mancano nella cache).
- **Soluzione hw: Chip Multithreading (CMT) / Hyper-Threading (SMT)**
 - L'hw assegna due o più thread hw (o CPU logiche) a un singolo core fisico.
 - Se un thread hw va in stallo in attesa della memoria, il core computa immediatamente su un altro thread hw, massimizzando l'uso del core.
 - Dal punto di vista del SO, ogni thread hw appare come una CPU logica separata.
- **Sfida per lo scheduling: due livelli di pianificazione**
 1. **Livello Software:** il SO decide quale thread software eseguire su quale thread hardware (CPU logica).
 2. **Livello Core (Hardware):** la logica hw interna al core decide quale thread hardware eseguire sul core fisico in un dato momento (es. round-robin o priorità dinamica).



- **Pianificazione consapevole:** se il SO è a conoscenza che i thread hardware sullo stesso core condividono risorse (come cache o pipeline), può prendere decisioni più efficaci, come assegnare due thread sw diversi a core distinti, anziché allo stesso core, per evitare che si rallentino a vicenda.

2. Sistemi NUMA (Accesso Non Uniforme alla Memoria)

L'architettura NUMA è una sfida di scheduling/gestione della memoria:

- **Architettura:** ogni processore fisico ha la sua memoria locale.

- **Accesso:** sebbene tutto lo spazio di indirizzamento sia condiviso, l'accesso di una CPU alla propria memoria locale è più rapido rispetto all'accesso alla memoria locale di un'altra PCU (memoria remota).
- **Sfida:** se un thread viene schedulato su una CPU distante dai dati che gli sono stati allocati, subirà un tempo di accesso alla memoria più lungo.
- **Soluzione:** gli algoritmi di scheduling e di gestione della memoria devono lavorare insieme. Devono allocare la memoria più vicina alla CPU su cui il thread è schedulato, per garantire l'accesso più veloce possibile.

Bilanciamento del carico

Il bilanciamento del carico (*load balancing*) è una tecnica essenziale nei sistemi SMP che mira a distribuire il carico di lavoro in modo uniforme tra tutte le unità elaborative. Questo è fondamentale per sfruttare al massimo i processori e impedire che alcuni restino inattivi mentre altri sono sovraccarichi.

- **Necessità:** il bilanciamento è tipicamente necessario solo nei sistemi in cui ciascun processore ha una coda privata di thread eseguibili. Nei sistemi con una coda comune, un processore inattivo preleva autonomamente il thread successivo, rendendo il bilanciamento superfluo.
- **Definizione di Carico Bilanciato:** può significare avere lo stesso numero di thread in ogni coda, o distribuire equamente le priorità dei thread.

Approcci per il bilanciamento

Esistono due metodi principali per ripartire i thread tra i processori:

1. **Migrazione Push:** un processo dedicato (lo scheduler) controlla periodicamente il carico dei processori e sposta i thread da un processore saturo ad altri più liberi o inattivi.
2. **Migrazione Pull:** un processore inattivo prende ("tira") un thread in attesa da un processore sovraccarico.

Questi due approcci non sono esclusivi e sono spesso applicati contemporaneamente in molti scheduler moderni.

Predilezione per il processore

La predilezione per il processore (*Processor Affinity*) descrive la tendenza di un processo ad avere una "preferenza" per il processore su cui è in esecuzione.

- **Vantaggio della cache calda (*warm cache*):** quando un processo è eseguito da un processore, i suoi dati più recenti rimangono nella memoria cache di quel processore. Se il thread viene schedulato nuovamente sullo stesso processore, può riutilizzare questi dati, accelerando l'accesso alla memoria. Questo è chiamato sfruttare una *warm cache*.
- **Costo della migrazione:** se un processo si sposta su un altro processore, la cache del processore di partenza deve essere invalidata e quella del processore di arrivo deve essere nuovamente riempita, il che comporta un costo elevato.

Tipi di predilizione

La predilizione può essere implementata in modi diversi:

- **Predilizione debole (*Soft Affinity*):** il SO tenta di mantenere il processo sul processore originale, ma non offre garanzie; il processo può migrare per motivi di bilanciamento.
- **Predilizione forte (*Hard Affinity*):** il SO garantisce che un processo non sarà spostato da un sottoinsieme di processori specificato.
- **Implicazioni delle code private:** l'uso di code private (una per ogni processore) garantisce automaticamente la predilizione per il processore.

Conflitto tra bilanciamento del carico e predilezione per il processore

Obiettivo	Azione Richiesta	Effetto sulla Cache/Memoria
Bilanciamento del Carico	Sposta i thread tra i processori.	Rimuove i vantaggi della <i>warm cache</i> e può aumentare il tempo di accesso alla memoria (in sistemi NUMA).
Predilezione (Affinity)	Mantiene il thread sullo stesso processore.	Sfrutta la <i>warm cache</i> e garantisce un accesso rapido alla memoria locale (in sistemi NUMA).

I moderni algoritmi di scheduling sono progettati per bilanciare questi obiettivi in conflitto in modo efficace.

Scheduling Real-Time della CPU

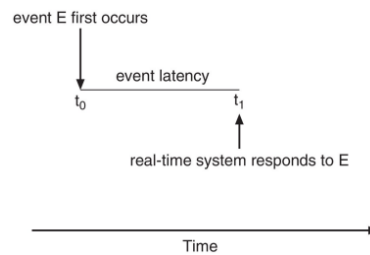
Lo scheduling nei sistemi real-time è unico perché i task devono essere eseguiti non solo correttamente, ma anche tempestivamente.

Si distinguono due tipi di sistemi:

1. **Real-Time Soft:** non offrono garanzie temporali, ma garantiscono che i processi critici abbiano precedenza sui processi non critici.
2. **Real-Time Hard:** hanno vincoli rigidi; i task devono essere completati entro una certa scadenza prefissata (*deadline*). Eseguirlo dopo tale scadenza è inutile.

Minimizzazione della latenza

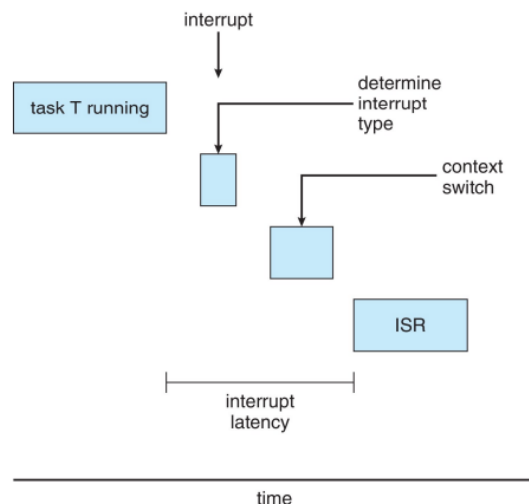
I sistemi real-time sono guidati dagli eventi, e la latenza relativa all'evento (*event latency*) è il tempo che intercorre tra l'occorrenza di un evento e il momento in cui il sistema inizia a gestirlo.



Due tipi di latenza devono essere minimizzati (e limitati superiormente nei sistemi hard):

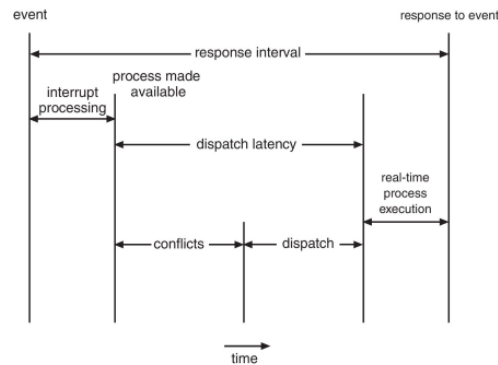
1. Latenza relativa alle interruzioni (*Interrupt Latency*)

- **Definizione:** il tempo tra la notifica di un'interruzione alla CPU e l'avvio della routine di servizio (ISR).
- **Fattori che la compongono:** completamento dell'istruzione corrente, determinazione del tipo di interruzione, salvataggio dello stato del processo corrente.
- **Minimizzazione:** i SO real-time devono inibire le interruzioni solo per intervalli di tempo molto piccoli.



2. Latenza di Dispatch (*Dispatch Latency*)

- **Definizione:** il periodo di tempo necessario al dispatcher per bloccare un processo e avviarne un altro.
- **Minimizzazione:** implementando kernel con prelazione.
- **Fase di conflitto:** comprende il tempo necessario per la prelazione di qualsiasi processo in esecuzione nel kernel e per la cessione delle risorse da parte di processi a bassa priorità richieste dal processo ad alta priorità.



Scheduling basato sulla priorità e caratteristiche dei task

I sistemi real-time utilizzano uno scheduler con prelazione basato sulla priorità.

- **Real-Time Soft:** è sufficiente uno scheduler con prelazione basato sulla priorità.
- **Real-Time Hard:** richiedono funzionalità aggiuntive per garantire che le scadenze siano rispettate.

Task Periodici (Real-Time Hard)

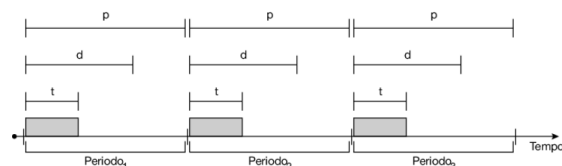
Per lo scheduling *hard*, i processi sono considerati **periodici**; richiedono la CPU a intervalli costanti.

Ogni processo periodico è definito da:

- **t:** tempo di elaborazione fisso;
- **d:** scadenza (*deadline*) entro cui deve completare l'esecuzione.
- **p:** periodo (intervallo di tempo).

La relazione tra il tempo di elaborazione, la scadenza e il periodo si può riassumere nelle disuguaglianze $0 \leq t \leq d \leq p$.

La frequenza di un processo periodico è data da $1/p$.

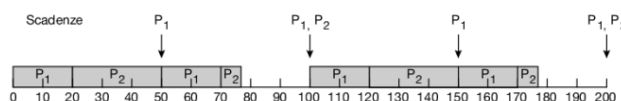


Lo scheduler può utilizzare un algoritmo di controllo dell'ammissione per decidere se accettare o rifiutare un task, garantendo di poterlo eseguire in tempo.

Algoritmi di Scheduling Hard Real-Time

Priorità proporzionale alla frequenza (*Rate Monotonic Scheduling - RMS*)

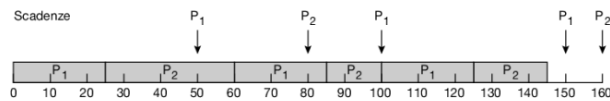
- **Tipo di priorità:** Statica con prelazione.
- **Regola:** la priorità è assegnata inversamente proporzionale al periodo ($1/p$). ($\Rightarrow$ più breve è il periodo, più alta la priorità)
- **Logica:** assegna priorità più alte ai processi che richiedono la CPU più frequentemente.
- **Ottimalità:** è un algoritmo ottimale nel senso che se non riesce a pianificare una serie di processi rispettando i vincoli temporali, nessun altro algoritmo che assegna priorità statiche vi riuscirà.
- **Limitazione:** l'utilizzo della CPU è limitato. Per N processi, l'utilizzo massimo garantito è $N(2^{1/N} - 1)$.



Scheduling EDF (Earliest-Deadline-First)

- **Tipo di priorità:** Dinamica.
- **Regola:** più vicina è la scadenza, maggiore è la priorità.

- **Funzionamento:** quando un processo diventa pronto, notifica la sua scadenza, e le priorità vigenti possono essere modificate.
- **Ottimalità:** è teoricamente ottimo; può garantire l'esecuzione entro le scadenze con un utilizzo della CPU fino al 100%.



Scheduling a quote proporzionali

- **Meccanismo:** distribuisce un numero totale di quote (T) tra tutte le applicazioni.
- **Garanzia:** un'applicazione con N quote è garantita di utilizzare la frazione N/T del tempo totale del processore.
- **Controllo:** richiede un meccanismo di controllo dell'ammissione per accettare solo richieste per le quali le quote disponibili sono sufficienti.

Scheduling Real-Time POSIX

Lo standard **POSIX.1b** fornisce un'estensione per l'elaborazione real-time, definendo due classi di scheduling real-time per i thread.

1. **SCHED_FIFO**: Pianifica i thread secondo la politica **FCFS** (*First-Come, First-Served*). **Non prevede time slicing** tra thread di pari priorità.
2. **SCHED_RR**: Adotta la politica **Round-Robin**. È simile a **SCHED_FIFO** ma **prevede il time slicing** tra thread di uguale priorità.

Le API per leggere e impostare la politica di scheduling sono `pthread_attr_getschedpolicy()` e `pthread_attr_setschedpolicy()`.

Criteri di scheduling nei SO moderni

Il termine scheduling dei processi viene utilizzato in senso generale, ma in realtà, nei sistemi moderni come Windows e Solaris, si descrive lo scheduling dei thread del kernel e in Linux lo scheduling dei task.

Scheduling di Linux (Completely Fair Scheduler - CFS)

La strategia predefinita di Linux si basa sul **Completely Fair Scheduler (CFS)**, introdotto nella versione 2.6.23.

Meccanismo (Scheduling Equo)

CFS non usa quanti di tempo fissi, ma assegna a ogni task una **percentuale del tempo di CPU**.

1. **Nice Value:** Stabilisce la percentuale di tempo CPU. L'intervallo è **da -20 a +19**; un valore **più basso** significa una priorità relativa **superiore**.
2. **vruntime (Tempo di Esecuzione Virtuale):** CFS traccia il tempo in cui ogni task è stato eseguito, modificato in base alla sua priorità.
 - **Task a priorità normale (nice=0):** `vruntime` coincide con il tempo effettivo.
 - **Task a priorità BASSA:** Il suo `vruntime` **aumenta più velocemente** del tempo effettivo.
 - **Decisione:** Lo scheduler seleziona sempre il task con il valore `vruntime` **più piccolo** (quello che ha avuto "meno tempo giusto").
3. **Struttura Dati:** I task eseguibili sono organizzati in un **albero Red-Black (R-B)**, bilanciato in base al valore `vruntime`. Il task con la priorità più alta (`vruntime` più piccolo) si trova sul lato sinistro dell'albero.

Scheduling real-time e classi

Linux usa classi di scheduling per assegnare priorità distinte.

Classe di Scheduling	Intervallo Priorità	Priorità Relativa	Note
Real-Time	0-99	Alta	Usa politiche POSIX (<code>sched_fifo</code> o <code>sched_rr</code>).
Normale	100-139	Bassa	Usa l'algoritmo CFS . Mappata dai Nice Value (es. -20 → 100, +19 → 139).

Bilanciamento del carico (NUMA-Aware)

CFS supporta il **bilanciamento del carico** usando una sofisticata tecnica che è anche **NUMA-aware** (consapevole dell'accesso non uniforme alla memoria).

- **Strategia Gerarchica:** Definisce **domini di scheduling**, insiemi di core raggruppati in base alla condivisione di risorse (es. cache L1, L2, L3).
- **Ordine di Bilanciamento:** Il bilanciamento avviene a partire dal **livello più basso** della gerarchia (evitando di spostare i thread più lontano dalla loro memoria locale, in particolare sui nodi NUMA distinti)

Scheduling di Windows

Windows utilizza un algoritmo di scheduling basato su **priorità e prelazione**.

Struttura delle Priorità

Lo scheduler di Windows (**Dispatcher**) usa uno schema a **32 livelli di priorità**.

- **Classe Real-Time:** Priorità **da 16 a 31**. Questi thread hanno l'accesso preferenziale alla CPU.
- **Classe Variabile:** Priorità **da 1 a 15**. La priorità dei thread in questa classe **può cambiare dinamicamente**.

Modifiche Dinamiche delle Priorità

Le priorità sono aggiustate per favorire i thread interattivi:

- **Riduzione:** Quando il quanto di tempo di un thread variabile si esaurisce, la sua priorità viene **ridotta** (ma mai sotto la **priorità di base**) per limitare l'uso della CPU da parte di thread *CPU-bound*.
- **Aumento:** Se un thread viene rilasciato da un'operazione d'attesa (specialmente per input interattivo come la tastiera), la sua priorità viene **aumentata** per garantire buoni tempi di risposta.
- **Foreground/Sfondo:** I processi in *foreground* (selezionati sullo schermo) ottengono un **aumento del quanto di tempo** (tipicamente moltiplicato per 3) per migliorare la reattività dell'utente.

Scheduling in Modalità Utente (UMS)

Windows 7 ha introdotto l'**User Mode Scheduling (UMS)**, che consente alle applicazioni di creare e gestire i propri thread in maniera efficiente, **senza l'intervento del kernel**.

Scheduling di Solaris

Solaris utilizza uno scheduling basato sulle priorità dove ogni thread appartiene a una delle sei **classi di scheduling**:

Classe di Scheduling	Tipo di Scheduling	Priorità Globale	Note
Real-Time (rt)	Prelazione / Fissa	Massima	Garanzia di risposta entro limiti prefissati.
Sistema (sys)	Fissa	Alta	Riservata ai thread del kernel (scheduler, demone paginazione).
Time-Sharing (ts) / Interattiva (ia)	Dinamica (Coda multilivello con retroazione)	Media	Relazione Inversa tra priorità e quanto di tempo (più alta priorità → meno tempo). Favorisce i processi interattivi (alta priorità dopo I/O).
Priorità Fissa (fp)	Fissa	Media-Bassa	Stesso livello di priorità di <i>ts</i> , ma non modificata dinamicamente.
Ripartizione Equa (fss)	Quote	Media-Bassa	Le decisioni sono prese sulla base delle quote (<i>shares</i>) di CPU assegnate a un insieme di processi (<i>progetto</i>).

Valutazione degli algoritmi di scheduling

La scelta di un algoritmo di scheduling è difficile perché ce ne sono molti, ognuno con parametri e prestazioni diverse.

Definizione dei criteri

Prima di valutare, è necessario definire i criteri di selezione e l'importanza relativa di ciascuna misura (es. utilizzo della CPU, tempo di risposta, produttività).

Metodi di valutazione analitica

La valutazione analitica fornisce una formula o un numero per valutare le prestazioni di un algoritmo dato un certo carico di lavoro.

Modellazione deterministica

- **Meccanismo:** considera un carico di lavoro predeterminato ed esatto. Si calcolano le metriche (es. tempo medio d'attesa) per ogni algoritmo.

- **Pro:** semplice, rapido e fornisce risultati numerici esatti.
- **Contro:** i risultati sono applicabili solo al carico di lavoro specifico considerato. Il suo uso principale è per la descrizione e gli esempi degli algoritmi.

Reti di code (Queueing Network Analysis)

- **Meccanismo:** descrive il sistema di calcolo come una rete di server con code d'attesa (es. CPU è un server con la *ready queue*).
- **Input:** si usano le distribuzioni di probabilità (es. esponenziali) dei tempi di arrivo e dei burst di CPU / I/O, anziché valori esatti.
- **Formula di little:** la relazione fondamentale $n = \lambda \times W$ (lunghezza media della coda n = tasso medio d'arrivo $\lambda \times$ tempo medio d'attesa W) è usata per calcolare le metriche.
- **Contro:** spesso richiede di fare assunzioni irrealistiche sulle distribuzioni di arrivo/servizio per mantenere la trattabilità matematica.

Metodi di valutazione empirica

1. Simulazioni

- **Meccanismo:** si programma un modello del sistema con strutture dati per rappresentare gli elementi chiave (CPU, code, processi). Un clock virtuale incrementa, modificando lo stato del sistema, e vengono raccolte statistiche.
- **Dati di Input:**
 1. **Generatore casuale:** usa numeri casuali basati su distribuzioni di probabilità (matematiche o empiriche).
 2. **Trace Tape:** è il metodo più preciso. Consiste nel monitorare il sistema reale e registrare la sequenza effettiva degli eventi, che viene poi usata come input per la simulazione.
- **Contro:** possono essere onerose e la progettazione del simulatore è complessa.

2. Realizzazione

- **Meccanismo:** l'unico metodo assolutamente preciso. L'algoritmo viene codificato e inserito nel SO, osservandone il comportamento in condizioni operative reali.
- **Costi/Difficoltà:** alto costo per codifica e test (su vm).
- **Cambiamento ambientale:** l'introduzione del nuovo algoritmo può modificare il comportamento degli utenti e dei programmi, che potrebbero tentare di "eludere" lo scheduler per ottenere prestazioni migliori (es. suddividendo processi lunghi in brevi se si dà priorità brevi).